UNIVERSIDAD NACIONAL DE LOJA

Ingeniería en Computación

Nombre: Martina Alejandra Maldonado Machuca

Docente: Ing. Luis Sinche

Sistemas Digitales: Apuntes de la Unidad 1

Semana 1: del computador al microcontrolador

1. ¿Qué es un sistema embebido?

Es cualquier aparato que tiene un microcontrolador adentro y hace una sola tarea fija. Tienen recursos limitados (poca memoria, gastan poca energía) y trabajan sin que uno se dé cuenta

Microprocesador vs. Microcontrolador

El microprocesador es solo la CPU: la memoria (RAM, disco) y la entrada/salida están en otros chips de la placa. Es lo que tiene un PC o una laptop, que son de propósito general. El microcontrolador, en cambio, mete todo en un solo chip: CPU, memoria Flash, SRAM y periféricos. Por eso es barato y consume poco, y se usa para tareas puntuales como una lavadora, un sensor o una alarma.

Cómo está armado un computador en sí, todo computador tiene tres piezas unidas por buses que son:

1. La CPU (ejecuta el programa)
2. La memoria (programa y datos)
3. Entrada/salida (pines, sensores, pantalla).

Los buses son tres, y cada uno responde una pregunta distinta. El bus de direcciones dice qué casillero de memoria (va de la CPU hacia afuera), el bus de datos lleva el contenido (va en los dos sentidos) y el bus de control indica si se lee o se escribe. La analogía del correo ayuda: la dirección es el sobre, el dato es la carta y la señal de control es el sello que dice "entregar".

Por dentro, la CPU tiene la ALU (suma, resta y hace AND, OR, XOR), los registros (casilleros internos muy rápidos; el AVR tiene 32, de r0 a r31), la unidad de control (lee cada instrucción y decide qué hacer) y el contador de programa (PC), que guarda la dirección de la siguiente instrucción. Un núcleo es una CPU completa: el ATmega328P tiene 1 y el ESP32 tiene 2.

La CPU repite siempre tres pasos al ritmo del reloj: buscar → decodificar → ejecutar.

Los tres tipos de memoria

La diferencia clave es qué pasa cuando se apaga el equipo y qué tanto se desgastan.

Memoria	¿Se borra al apagar?	Para qué sirve	Analogía
Flash	No	Guarda el programa; se reescribe al subir código desde el IDE	Un libro
SRAM	Sí	Guarda las variables mientras el programa corre; rápida y pequeña	Una pizarra
EEPROM	No	Pocos datos que el programa cambia y deben sobrevivir al apagado	Una libreta

Arquitectura: Von Neumann vs. Harvard

- **Von Neumann:** Usa un solo bus para traer instrucciones y datos por turnos (como compartir la misma vía).
- **Harvard:** Tiene **buses y memorias separadas** para el programa y para los datos. El **ATmega328P** usa esta, lo que la hace más eficiente porque lee la siguiente instrucción mientras procesa un dato.

Operaciones a nivel de bits y máscaras

Para no usar funciones lentas y controlar el hardware directo, tocamos los bits de un registro con compuertas lógicas usando una máscara (un número con un 1 en la posición que nos interesa, ej: 1 << n):

OR (|=): Para encender un bit específico (ponerlo en 1) sin tocar los demás.

AND (&= ~): Para apagar un bit (ponerlo en 0).

XOR (^=): Para invertir un bit**.**

AND (& en un if): Para preguntar si un bit está encendido.

Bits, hexadecimal y operaciones bit a bit

Un bit es un nivel de voltaje: el 0 es cerca de 0 V y el 1 es cerca de 5 V en el Arduino Uno o de 3,3 V en el ESP32. Un byte son 8 bits, numerados de derecha a izquierda (bit 0 a la derecha, bit 7 a la izquierda), y su valor es la suma de los pesos (128, 64, 32, 16, 8, 4, 2, 1) de los bits que están en 1. Un dígito hexadecimal representa 4 bits, así que 0010 0000 es 0x20. En C, `0x` indica hexadecimal y `0b` binario.

Las operaciones bit a bit se hacen columna por columna, cada una independiente: AND da 1 solo si ambos son 1, OR da 1 si alguno es 1, XOR da 1 si son distintos y NOT invierte. La CPU hace las 8 columnas a la vez en una sola instrucción; por ejemplo, `0xCA & 0x0F` da `0x0A`.

Para tocar un solo bit sin afectar los demás se usa una máscara: `1 << n` es un byte con un 1 solo en el bit n.

Donde la máscara tiene 0, el bit no cambia. Con eso salen las cuatro recetas:

Quiero	En C	Por qué funciona
Poner el bit n en 1	`REG |= (1 << n);`	OR con 1 activa el bit; OR con 0 lo deja igual.
Poner el bit n en 0	`REG &= ~(1 << n);`	~ invierte la máscara y AND con 0 apaga el bit.
Invertir el bit n	`REG ^= (1 << n);`	XOR con 1 invierte el bit; XOR con 0 lo deja igual.
Preguntar por el bit n	`if (REG & (1 << n))`	El resultado es distinto de 0 solo si el bit está en 1.

Un error común es escribir `REG = (1<<2);` para encender el bit 2: eso asigna el registro completo y apaga todos los demás bits. Hay que usar `|=`.

ATmega328P vs. ESP32, y algo de sostenibilidad

El ATmega328P (Microchip, antes Atmel; placa Arduino Uno) es de 8 bits, un núcleo, 16 MHz, pines a 5 V y sin Wi-Fi ni Bluetooth. El ESP32 (Espressif; placa DevKit V1) es de 32 bits, dos núcleos, hasta 240 MHz, pines a 3,3 V (no tolera 5 V) y trae Wi-Fi y Bluetooth. Conviene distinguir chip, módulo y placa: el chip ESP32 va en el módulo ESP32-WROOM-32 (con la Flash y la antena bajo una tapa metálica), que a su vez va en la placa DevKit V1 (USB, regulador, botones EN y BOOT, pines). Y Arduino no es un chip, es una plataforma (placas, IDE y funciones como `pinMode`), que también sirve para programar el ESP32. En sostenibilidad la idea es elegir el MCU mínimo suficiente: si no necesito Wi-Fi, uno sin radio consume menos; sobredimensionar pines y memoria gasta energía y material; y si va con batería cada miliamperio cuenta. Menos recursos significa menos consumo y menos residuos electrónicos, y los proyectos deben justificar esa elección.

Semana 2: del AVR al ESP32

El salto de 8 a 32 bits

La analogía de la clase fue pasar de la tienda de barrio al supermercado: 32 bits en vez de 8, dos núcleos Xtensa LX6 hasta 240 MHz en vez de uno a 16 MHz, 520 KB de SRAM en vez de 2 KB, 4 MB de Flash en un chip aparte del módulo (por SPI) en vez de 32 KB dentro del chip, y lógica de 3,3 V en vez de 5 V. Otra diferencia es que en el AVR cada pin está cableado a un periférico fijo (PB5 es el bit 5 de PORTB), mientras que en el ESP32 la asignación es configurable a través de la GPIO matrix y el IO_MUX, que funcionan como una central telefónica entre los pines y los periféricos (UART, PWM, SPI, GPIO simple). Configurar un pin son tres pasos (elegir su función en el IO_MUX, conectar la señal en la matrix y habilitar la salida) y pinMode() los hace todos. Lo que no cambia es que un pin se controla escribiendo bits en registros mapeados en memoria.

Equivalencias de registros

Función	AVR (semana 1)	ESP32 (GPIO 0–31)
Dirección (entrada/salida)	DDRB	GPIO_ENABLE_REG
Escribir la salida	PORTB	GPIO_OUT_W1TS_REG / GPIO_OUT_W1TC_REG
Leer la entrada	PINB	GPIO_IN_REG
Pull-up interna	PORTB (con el pin como entrada)	registro IO_MUX de cada pin

Los GPIO 32 a 39 no caben en un registro de 32 bits, así que usan un segundo banco (GPIO_OUT1_REG, GPIO_IN1_REG).

Registros y atomicidad (W1TS y W1TC)

En el AVR usábamos DDR PORT y PIN. En el ESP32 la estructura cambia y usa registros de habilitación y salida. Como el ESP32 tiene dos núcleos y pueden ocurrir choques, usa registros especiales de escritura atómica: W1TS (escribes un 1 para encender un bit) y W1TC (escribes un 1 para apagarlo) sin necesidad de leer lo que había antes, evitando que se pierdan datos.

Pines seguros, strapping y límites eléctricos

- En la DevKit V1 de 30 pines se debe revisar la serigrafía, ya que el orden puede variar entre clones. Los GPIO 6–11 están reservados para la memoria Flash, los GPIO 34, 35, 36 y 39 son solo de entrada, y los GPIO 1 y 3 se utilizan para la comunicación UART0. Para LEDs y pulsadores se recomiendan los GPIO 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32 y 33.
- Los pines de strapping determinan el modo de arranque del ESP32. Por eso, se debe evitar conectar entradas que alteren su estado inicial, especialmente en los GPIO 0, 2, 5, 12 y 15. Estas fallas pueden presentarse en la placa real, aunque Wokwi no las simule.
- Finalmente, los GPIO trabajan a 3,3 V y no toleran 5 V. Se recomienda limitar la corriente a 10 mA por pin y utilizar LEDs rojos, verdes o amarillos con resistencias de 220 a 330 Ω para evitar daños.

Tipos de datos estrictos (stdint.h)

Un int no mide lo mismo en todas partes (en un Arduino viejo es de 2 bytes, en el ESP32 de 4). Por eso siempre debemos usar tipos con el tamaño exacto en el nombre: uint8_t (8 bits sin signo), uint32_t (32 bits), etc., para evitar desbordamientos locos en los cálculos.

Tiempo sin bloquear (millis() vs delay())

- **delay()** es el peor enemigo de un programa interactivo porque congela la CPU por completo y si presionas un botón en ese momento, el microcontrolador ni se entera (se pierde la pulsación).

- **millis()** solo mira el "reloj de pared" del sistema. Restando el tiempo actual con uno guardado (ahora - tAnterior >= PERIODO), podemos hacer tareas repetitivas o parpadear LEDs sin bloquear el código, dejando libre la CPU para leer botones.

Pulsador: pull-up, rebote y flanco

Con INPUT_PULLUP, el pin lee HIGH cuando el botón está suelto y LOW cuando está presionado. Los GPIO 34–39 no tienen resistencia pull-up interna. Como los pulsadores generan rebotes (cambios rápidos y falsos), se aplica un antirrebote de 30 ms, aceptando el cambio solo si la señal permanece estable. Se evita usar delay(), porque bloquea el programa. También es importante diferenciar entre leer por nivel y por flanco. Para contar una sola vez cada pulsación, se detecta únicamente el cambio de HIGH a LOW, evitando que el contador aumente repetidamente mientras el botón está presionado. El programa se organiza en tres bloques dentro de loop():

1. **Entradas**: lectura del botón con antirrebote y detección de flanco.
2. **Lógica**: control de tiempos mediante millis().
3. **Salidas**: encendido de LEDs según el estado del programa.

<u>Semana 3:</u> temporizadores de hardware

Qué es un temporizador

Un temporizador (timer) es un circuito dentro del microcontrolador que cuenta pulsos de reloj. Como cada pulso dura un tiempo conocido, sirve para medir tiempo y para avisar cuando pasó un tiempo determinado. No es un programa sino hardware, un periférico aparte de la CPU, como los GPIO. Con delay() la CPU se queda esperando, con millis() pregunta "¿ya pasó?" en cada vuelta (si el loop() se ocupa 40 ms, el aviso llega con hasta 40 ms de retraso) y con el temporizador un circuito aparte cuenta y avisa a la hora exacta.

¿Cómo funciona por dentro?

Componente	Función principal en el ESP32
Reloj Base (APB)	Proporciona una frecuencia fija de 80 MHz (80 millones de pulsos por segundo).
Prescaler	Actúa como un divisor de frecuencia para que el contador avance más lento y se ajuste a nuestras necesidades de tiempo.
Alarma	Es un número límite que se guarda en un registro; cuando el contador lo alcanza, se genera un aviso automático.
Autorrecarga (Auto-reload)	Hace que el contador regrese a cero de forma automática apenas alcanza la alarma, repitiendo el ciclo sin intervención del programa.

Reloj base, prescaler y resolución

Los temporizadores del ESP32 reciben un reloj APB de 80 MHz. Como esta frecuencia es muy alta, se utiliza un prescaler para dividirla y facilitar la medición del tiempo. Por ejemplo, con un divisor de 80 se obtiene 1 tick = 1 µs. Esto permite controlar la precisión y el tiempo que puede medir el contador. La resolución es el tiempo que dura cada tick y se calcula como 1/f. Finalmente, la alarma establece un valor límite que, al ser alcanzado por el contador, genera un aviso automático.

tiempo = alarma × resolución alarma = tiempo × f N = 80 000 000 / f

- **Ejemplo 1:** aviso cada 0,5 s con f = 1 000 000. La resolución es 1 µs, la alarma es 0,5 × 1 000 000 = 500 000 y el prescaler es N = 80.
- **Ejemplo 2:** f = 10 000 y alarma = 2500. La resolución es 100 µs, el tiempo es 2500 × 100 µs = 0,25 s (4 avisos por segundo) y N = 8000, que es válido.

En la actividad en parejas salieron: con f = 1 MHz y alarma 2000 el aviso es cada 2 ms; para 10 ms con f = 1 MHz la alarma es 10 000; y con f = 40 000 y alarma 40 000 el aviso es cada 1 s y N = 2000.

Autorrecarga y desbordamiento

El comparador verifica en cada tick si el contador alcanzó la alarma. Al hacerlo, genera un evento y, si está activada la autorrecarga (auto-reload), el contador vuelve automáticamente a 0 y repite el ciclo. No se debe confundir la alarma, que es un límite establecido por el programador, con el desbordamiento, que ocurre cuando el contador alcanza su capacidad máxima de bits y vuelve a 0.

El máximo de un contador de n bits es \(2^n-1\). Por ejemplo, uno de 8 bits llega hasta 255, uno de 16 bits hasta 65 535 y uno de 64 bits hasta un valor enorme. Por eso, en el ESP32 se trabaja principalmente con alarmas y no con desbordamientos.

Los temporizadores del ESP32 frente al AVR

Característica	ESP32 (WROOM-32)	ATmega328P (Uno)
Temporizadores de uso general	4 (2 grupos, TIMG0 y TIMG1, de 2 timers cada uno)	3 (Timer0, Timer1, Timer2)
Tamaño del contador	64 bits	8 o 16 bits
Reloj de entrada	80 MHz	16 MHz
Prescaler	Cualquier N de 2 a 65 535 (16 bits)	Solo 1, 8, 64, 256 o 1024
¿Se llena (sin prescaler)?	Prácticamente nunca	Timer1: en 4,1 ms

Cada timer del ESP32 puede contar hacia arriba o hacia abajo y tiene autorrecarga, y como funcionan en paralelo se pueden tener hasta 4 avisos periódicos independientes.

API, core y versión 3.x

Una API es un conjunto de funciones ya programadas que permiten utilizar el hardware sin conocer su funcionamiento interno. Por ejemplo, pinMode(), digitalWrite(), delay() y millis(). En los temporizadores, la API configura automáticamente los registros, el prescaler y la alarma.

El Core es el paquete que permite al Arduino IDE programar una placa específica. Para utilizar el ESP32 se debe instalar ESP32 by Espressif Systems desde el Gestor de placas. Es importante considerar que en la versión 3.x cambiaron las funciones y parámetros de los temporizadores respecto a la versión 2.x, por lo que algunos códigos de tutoriales antiguos pueden generar errores de compilación.

Interrupciones, callback, IRAM_ATTR, volatile y bandera

- Una **interrupción** es una señal que avisa a la CPU que ocurrió un evento. La CPU pausa momentáneamente su tarea, ejecuta una función especial llamada ISR y luego continúa. Puede ser interna, como la de un temporizador, o externa, como la de un pulsador.

- Un **callback** es una función que se entrega al temporizador para que este la ejecute automáticamente cuando se cumpla la alarma. Esta función es la ISR y se identifica con IRAM_ATTR, que permite ubicarla en la memoria interna IRAM para su ejecución.

Las variables compartidas entre la ISR y loop() deben declararse como volatile, para asegurar que el programa siempre lea su valor actualizado. Finalmente, se utiliza el patrón bandera (flag) para evitar tareas largas dentro de la interrupción: la ISR únicamente activa una bandera (true) y loop() la detecta, la desactiva y realiza el trabajo necesario, como imprimir mensajes.

Semana 4: Interrupciones externas en el ESP32

En lugar de hacer sondeo (*polling*), que es estar preguntando en cada vuelta del loop() si ya presionaron un botón (con el riesgo de que si el programa está ocupado en un delay se pierda el toque), usamos una interrupción externa. Aquí es un circuito físico de los pines GPIO el que vigila el botón y avisa de inmediato a la CPU cuando ocurre el cambio.

Para atrapar el evento usamos flancos configurados con attachInterrupt():

- **FALLING:** Cuando la señal cae de 1 a 0 (ideal para botones con INPUT_PULLUP que al presionarse van a GND).

- **RISING:** Cuando la señal sube de 0 a 1 (al soltar).

- **CHANGE:** Cuando ocurre cualquiera de los dos.

La función que atiende la interrupción se llama ISR (Interrupt Service Routine) y tiene reglas súper estrictas para no crashear la placa:

1. **Debe ser cortísima** (solo sube una bandera o suma un contador). Nada de Serial.print ni delay, porque si te pasas, el *Watchdog* detecta que el microcontrolador se trabó y reinicia la placa tirando un error (*Guru Meditation Error*).

2. **Usa IRAM_ATTR**: Toda ISR debe llevar esta etiqueta antes del nombre (void IRAM_ATTR miISR()) para que el código viva en la memoria IRAM rápida y nunca falle si la caché se apaga momentáneamente.

3. **Usa volatile:** Las variables globales que se comparten entre la ISR y el loop() deben declararse como volatile (ej. volatile bool alerta = false;), de lo contrario el compilador optimiza el código guardando copias y el bucle principal nunca se entera de que la interrupción cambió el valor.

4. **Patrón bandera:** La ISR solo pone una bandera en true y sale rápido. Luego, en el loop(), revisamos la bandera con calma, la bajamos a false, y hacemos el trabajo pesado (como imprimir o encender cosas).

Por último, como un botón mecánico produce falsos contactos rápidos al presionarlo (rebote), la interrupción corre varias veces por un solo toque. Para solucionarlo, agregamos un filtro de tiempo usando millis() dentro de la propia ISR para ignorar flancos que ocurran a menos de 50 ms del último aceptado.